Page 1: System Overview & Architecture Context

Introduction

The PrecisionForge project integrates an advanced Python-based neural memory agent (the **PCAM Precision Agent**) with a cinematic, scroll-driven web interface. The system allows users to seamlessly trigger inference-time memory steering evaluations via a local, live-running benchmark harness.

High-Level Architecture

The system is divided into three major architectural components that communicate locally:

1. **The Core Intelligence (Python)**
    - Houses the mathematical algorithms, neural memory models (PCAM), and the `Engine` adapter (`w/adapters/myteam.py`).
    - Executes strictly deterministic precision control over memory retrieval.
2. **The API Gateway (Node.js/Express)**
    - Acts as the bridge between the browser UI and the heavy Python processes.
    - Located at `site/server.js` , it safely spawns Python child processes, monitors execution streams, and normalizes output into structured JSON format.
3. **The Presentation Layer (React/Vite)**
    - A highly optimized, cinematic frontend integrating Spline 3D graphics, Framer Motion animations, and Lenis smooth scrolling.
    - Built to handle long-polling or asynchronous waits seamlessly (e.g., waiting 77 seconds for a benchmark to complete).

Data Flow

When a user clicks "Run Benchmark" on the frontend:

1. The Vite React client fires a GET request to the Express API (`/api/benchmark`).
2. Express spawns a local `python3` process pointing to `bench_p04_pcam/self_check.py` .
3. The Python harness imports `w.adapters.myteam:Engine` dynamically and begins evaluating the PCAM environment across multiple seeds.
4. Python flushes its final results to `stdout` .
5. Express captures `stdout` , runs regex parsing to extract key metrics (Accuracy, Speed, Points), and returns a JSON payload.
6. The React UI animates the results into view using `StatCounter` components.

Page 2: The Core Intelligence (Python Engine)

The PCAM Harness

Located in the `bench-p04-pcam` directory, the benchmark harness orchestrates the evaluation of the memory agent. It loads synthetic stored patterns, injects noise/corruption into query vectors, and scores the agent based on retrieval accuracy and anisotropy spread reduction.

The Custom Adapter (`myteam.py`)

At the heart of the system is the `Engine` class, a deterministic precision controller that relies on NumPy. It avoids retraining or external state, operating purely at inference time.

The Two-Regime Strategy

The agent fundamentally splits incoming queries into two processing regimes based on a nearest-pattern cosine confidence score:

1. **Retrieval Regime (Corrupted Queries)**
    - Triggers when confidence is low (`<= 0.82`).
    - Computes a corruption map by comparing query magnitude against stored-pattern scales.
    - Allocates higher precision weights to masked dimensions, forcing the PCAM gradient to rapidly refill the missing signal.
    - Applies a small class-conditional shape prior to guide convergence.
2. **Geometry Regime (Near-Clean Probes)**
    - Triggers when confidence is high (`>= 0.90`).
    - Operates near the true equilibrium state rather than the raw input vector.
    - Computes the live Hessian matrix H and determines the optimal diagonal preconditioner Π to minimize the eigenvalue spread (condition number) of $\Pi^{1/2} H \Pi^{1/2}$.
    - Resolves the quasi-convex objective using subgradient projection with four analytic warm starts (Identity, Jacobi, Half-Jacobi, and Inverse-Curvature).

This intelligence allows the memory model to "steer itself" without backpropagation.

Page 3: The Web Presentation Architecture

UI Philosophy

The frontend (`site/`) discards traditional SaaS dashboard aesthetics in favor of an immersive, editorial, and cinematic experience.

Key Technologies

- **Vite & React 19**: Lightning-fast local development and strict component architecture.
- **Framer Motion**: Drives all entrance animations, scroll-triggered reveals (`whileInView`), and continuous SVG loops.
- **Lenis Smooth Scroll**: Overrides native browser scrolling to provide a buttery-smooth, interpolated timeline that enhances the physical feel of the site.
- **Spline 3D**: Integrates a live, interactive 3D WebGL scene directly into the Hero section without requiring heavy custom Three.js implementations.

Component Hierarchy

The UI is strictly modularized within `site/src/components/` :

- **`<App />`** : The root orchestrator. Initializes the Lenis RequestAnimationFrame loop and stacks the sequential components.
- **`<Hero />`** : The entry point. Merges the Spline background with dark glass overlays and dynamic `Playfair Display` typography.
- **`<Nav />`** : A stateful, scroll-aware navigation bar that remains completely invisible until the user scrolls past the 90vh mark, fading in to anchor the experience.
- **`<Problem />` & `<HowItWorks />`** : Educational sections that use staggered grid layouts and animated SVG diagrams to explain the Python agent's two-regime theory.
- **`<Results />`** : A high-impact display utilizing a custom `<StatCounter />` component. As the user scrolls into view, the counters animate dynamically to their final scores (e.g., 70/70 and 73.05).
- **`<Benchmark />`** : The interactive execution card. Connects the frontend to the Node.js API, managing the loading state, the `fetch` lifecycle, and displaying the parsed JSON results via dynamic color-coded mini-cards.

Conclusion

The architecture cleanly decouples the heavy computational AI layer from the sleek presentation layer, using Node.js as a robust intermediary. This allows the system to remain highly responsive on the client while managing intensive 70-second benchmark tasks under the hood.